Chat session - Tech stack recommendation

USER

> Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team. Structure the stack towards developing a dynamic website with a realtime database.
>
> You are allow to refer to and only to the following documents in the codebase:
>
> - SRS document + Usecase diagram lab2
> - Refined class diagram and dialog map lab3
> - Mockup UI: you may only refer to the corresponding UI prototype of current implementation in src

MODEL - GPT-6.1-Sol (high thinking effort)

▸ 6 previous messages (collapsed)

Recommended stack

Keep the React prototype and build around Firebase Realtime Database. A single page app is a good fit for Vibecheck's signed-in dashboard, questionnaire, profile, and recommendation screens. It lets an entry-level team extend the existing UI without introducing a second web framework. The SRS requires saved assessment results, personalized games and gamer matches, and later behavior monitoring; the refined dialog map marks matching and monitoring as planned. (source: Gaming Project SRS.pdf) Lab 3 dialog map

Layer	Choice	Why it fits
Frontend	**React, React Router, JavaScript, and Vite**	Continues the existing JSX pages and routes in App.jsx. Use the existing CSS while the UI is being completed.
Sign-in	**Firebase Authentication**	The prototype already uses it for account creation, login, and protected routes.
Primary data store	**Firebase Realtime Database**	Its listeners suit live profile, assessment, friend-request, and notification views. The prototype already listens to account and result data in useAccountProfile.js and useQuestResult.js. Listen at specific paths so updates remain small. Firebase guidance.
Server logic	**Cloud Functions for Firebase, using JavaScript**	Put Steam linking, RAWG requests, gamer matching, and account deletion behind authenticated endpoints. This keeps API secrets and private matching data out of the browser; callable functions can receive Firebase Auth credentials. Callable functions, secret storage.
Hosting and local development	**Firebase Hosting and the Firebase Emulator Suite**	Hosting supports the app's routes through an SPA rewrite; emulators let the team check Auth, database rules, and functions locally. Hosting, emulators.

First architecture change: choose Realtime Database as the authoritative store for gamer profiles. The prototype currently saves that profile in Firestore through profileStore.js, while account and quest data use Realtime Database. Migrating the profile once will give the team one data model to maintain. Keep browser storage for unfinished questionnaire progress and temporary display caches.

Use paths such as `users/{uid}/profile`, `users/{uid}/questResults/latest`, and `users/{uid}/savedGames`. Keep private user data under owner-only database rules; expose only an explicitly opted-in matching profile. Calculate matches in a function that checks consent and blocks before returning results. This follows the SRS privacy rules and the matching entities in the refined class diagram. Firebase rules guidance.

Build order: finish the assessment-to-profile data flow; move RAWG requests out of GameRecc.jsx and enforce age and content eligibility there; add opt-in matching; then implement monitoring. The current recommendation page still has a demo profile and a browser-side RAWG key, so it should not be treated as the finished data flow. The use cases call for recommendations and matches after a completed profile, while monitoring is a later feature in the refined design. (source: Use Cases, Use case model, UI Mockup & Class Diagram.pdf)

Answering question 3.3.2

The two runs are mostly different for the following points:

1. We didn't document the model specification in the earlier run, so it may have used a stronger or weaker model, or a different reasoning effort.
2. The UI mock-up has been replaced by a more mature implementation than in the earlier run.
3. The earlier run didn't have access to the class diagram or dialog map.

Since the AI in this run was given more context and a more complete implementation, it did not give generic suggestions such as styling or API style, but focused on developing the existing work.

Overall tech stack comparison.

Layer	Lab 2 recommendation	Lab 3 recommendation
Frontend	React + TypeScript + Vite, Tailwind CSS	React + React Router + JavaScript + Vite, existing CSS
Authentication	JWT + bcrypt	Firebase Authentication
Database	PostgreSQL + Prisma	Firebase Realtime Database
Server logic	Node.js + Express + TypeScript (REST)	Cloud Functions for Firebase (JavaScript)
Hosting	Vercel, Render/Railway, Supabase/Neon	Firebase Hosting + Emulator Suite

Both runs still keep the architecture simple, suit an entry-level team, and call external game APIs from the server rather than the browser.

1. **Server-side work.** The diagram has `SteamLinkController` on our Express server, plus `SteamApiAdapter` and `RAWGApiAdapter`. The AI kept Steam linking and RAWG requests on a secure server, but suggested Cloud Functions instead of Express so that everything runs on one platform.

2. **Language.** Operations use JavaScript names from our code and return `Promise<T>` for asynchronous calls. The AI recommended plain JavaScript instead of TypeScript.

3. **Planned features.** Gamer Matching and Behaviour Monitoring are labelled **(planned)**, and the matching classes filter blocked users and users who have not enabled matching. The AI scheduled those features last and suggested a matching function that checks consent and blocks, behind owner-only database rules.

Chat session - Minimal skeleton

User

> Using only class_diagram.plantuml as input, generate a minimal skeleton template for our project.
>
> Do not read or copy anything from vibecheck/, lab1/ or lab2/. Do not modify any existing files.
>
> Technology: React with Vite in plain JavaScript (.jsx for UI components) for the frontend, and a small Node.js + Express server for the server-side classes.
>
> Rules:
> 1. Create a folder hierarchy and files only, all inside `@lab3/ai_skeleton/`
> 2. Every file must be empty except for TODO comments describing what that file will contain, based on the class diagram.
> 3. Do not write any code: no imports, exports, functions, classes, JSX, variable declarations or configuration values.
> 4. Do not create package.json or any other configuration files.
> 5. Choose a folder structure based on the class diagram and its stereotypes (boundary, controller, entity).
>
> When you have finished:
> - show the full folder tree
> - explain why you chose this structure
> - list which file each class maps to, and any class you did not give a file, with the reason
> - create lab3/AI-generated skeleton (3.3.3).zip containing only lab3/ai_skeleton/

MODEL - GPT-6.1-Sol (high thinking effort)

▸ 6 previous messages (collapsed)

Created the TODO-only skeleton and ZIP archive. The ZIP contains only `lab3/ai_skeleton/`.

```
lab3/ai_skeleton/
├── frontend/
│   ├── index.html
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       ├── boundaries/
│       │   ├── account-steam/UpdateProfileUI.jsx
│       │   ├── assessment/
│       │   │   ├── ProfileResultUI.jsx
│       │   │   └── QuestionnaireUI.jsx
│       │   ├── authentication/
│       │   │   ├── FirebaseAuthAdapter.js
│       │   │   ├── LoginUI.jsx
│       │   │   ├── ProtectedRoute.jsx
│       │   │   └── SignupUI.jsx
│       │   ├── behaviour-monitoring/
│       │   │   ├── BehaviourMonitoringUI.jsx
│       │   │   └── NotificationUI.jsx
│       │   ├── dashboard/HomeUI.jsx
│       │   ├── game-recommendation/
│       │   │   ├── GameDetailUI.jsx
│       │   │   └── GameRecommendationUI.jsx
│       │   └── gamer-matching/
│       │       ├── GamerDetailUI.jsx
│       │       └── GamerMatchingUI.jsx
│       └── controllers/
│           ├── account-steam/UserProfileController.js
│           ├── assessment/QuestController.js
│           └── authentication/AuthController.js
├── server/
│   └── src/
│       ├── server.js
│       ├── boundaries/
│       │   ├── account-steam/SteamApiAdapter.js
│       │   └── game-recommendation/RAWGApiAdapter.js
│       └── controllers/
│           ├── account-steam/SteamLinkController.js
│           ├── behaviour-monitoring/BehaviourMonitoringController.js
│           ├── game-recommendation/GameRecommendationController.js
│           └── gamer-matching/GamerMatchingController.js
└── shared/
    └── entities/
        ├── User.js
        ├── account-steam/
        │   ├── AccountProfile.js
```

```
        │   ├── OwnedGame.js
        │   ├── PlaytimeSnapshot.js
        │   ├── SteamAccount.js
        │   └── UserProfile.js
        ├── assessment/
        │   ├── Answer.js
        │   ├── Choice.js
        │   ├── Encounter.js
        │   ├── QuestResult.js
        │   └── TraitScore.js
        ├── behaviour-monitoring/
        │   ├── BehaviourSummary.js
        │   ├── GamingActivity.js
        │   ├── Notification.js
        │   └── WarningRule.js
        ├── game-recommendation/
        │   ├── Game.js
        │   ├── GameRecommendation.js
        │   ├── Genre.js
        │   ├── InteractionLogEntry.js
        │   ├── Platform.js
        │   └── SavedGame.js
        └── gamer-matching/
            ├── FriendRequest.js
            └── GamerMatch.js
```

Structure: Diagram boundaries that display UI are under the React frontend; browser session and local-state controllers are there too. Steam and RAWG adapters and the server-oriented controllers are under Express. Entities are shared and grouped by the diagram's feature packages.

Class mapping: Each of the diagram's 46 classes maps to the identically named `.jsx` or `.js` file in the tree above. No class was omitted. `index.html`, `App.jsx`, `main.jsx`, and `server.js` are TODO-only entrypoint placeholders, not diagram classes. I verified all 50 files contain only TODO comments and the ZIP contains those 50 files under the requested folder.

Answering question 3.3.4

We checked the output ourselves instead of relying on the AI's summary. The class diagram declares 46 classes, and each has exactly one file. All 50 files contain only `// TODO:` or `<!-- TODO: -->` lines. No existing files were changed. We also checked that the agent correctly followed the instruction to read only the class diagram and not touch the existing code structure.

To what extent do we agree with its choices? We mostly agree with the `frontend/`, `server/` and `shared/` structure. Our own code structure at the time of this lab is similar, with `src/` and `server/`.

Overall, we disagree with two choices:

1. **AI puts `GameRecommendationController` in the server.** Several of its operations work with the browser: `recommendationKey` and `readRecommendations` handle the local cache, and `saveFavorites` and `logInteraction` are client-side. Only the RAWG calls need the server. The diagram does not say where each controller runs, so the AI guessed.
2. **Entities are one file per class in `shared/`.** That is a Java-style mapping. In our JavaScript app, entities are plain objects stored in Firebase.

Has it missed something obvious or failed to produce anything reasonable? It produced a reasonable, complete mapping of the classes, but it missed everything the diagram does not show:

- **No persistence layer.** There is no file to initialise Firebase or read and write the Realtime Database. Our diagram has `FirebaseAuthAdapter` but no class for database access, so this is a gap in our own design model, not just in the AI's output.
- **No access-control file.** There is no `database.rules.json`. Our prompt banned configuration files, so this omission comes partly from our own instructions.

Did it add anything beyond what was requested? No implementation code was added to any file. It did add four files that are not diagram classes: `index.html`, `main.jsx`, `App.jsx` and `server.js`. These are TODO-only entry points, and the AI disclosed them. To create the skeleton, the AI wrote and ran a Node.js generator script in its own terminal. That code is not in the output, but it explains why the comments are so mechanical. Its first run also missed `GameRecommendationUI` and wrote a wrong comment in `User.js`. The AI caught and fixed both with its own check before creating the .zip, which shows why its output still needs review.